

Construção de uma Aplicação para Geração de Autômatos Finitos Determinísticos Minimizados

Autores: Luan Lucas de Lima Peloso

Disciplina: Linguagens Formais e Autômatos – Ciência da Computação (UFFS)

1. Resumo

Este trabalho apresenta a especificação e implementação de uma aplicação em Python capaz de gerar um Autômato Finito Determinístico (AFD) a partir de um arquivo de texto contendo tokens e Gramáticas Regulares (GRs). A solução realiza a conversão de um Autômato Finito Não-Determinístico (AFND) inicial para um AFD equivalente e, em seguida, aplica um processo de minimização com a remoção de estados inacessíveis e mortos, finalizando com a inclusão de um estado de erro para as transições vazias. A implementação utilizou estruturas nativas da linguagem, como dicionários e conjuntos matemáticos, assegurando eficiência nas transições de estados e respeito rigoroso às restrições do projeto, como a abstenção do uso de classes de equivalência.

2. Introdução

Na teoria de processamento de linguagens, o bloco léxico é fundamental para ler uma cadeia de caracteres de entrada e transformá-la em tokens agrupados logicamente. O fundamento matemático responsável por essa capacidade de reconhecimento léxico é a teoria de autômatos.

Nesse contexto, o presente projeto tem como objetivo a construção de uma aplicação para gerar um AFD livre de estados inalcançáveis e mortos a partir de um arquivo fonte contendo sentenças na forma de cadeias ou Gramáticas Regulares (Notação BNF). A complexidade do problema exige a construção primária de um autômato não-determinístico (onde estados intermediários não são reaproveitados entre regras) , exigindo a posterior aplicação computacional de teoremas de determinização e minimização estrutural.

3. Referencial Teórico Básico

3.1. Autômatos Finitos (AF)

Um Autômato Finito é um tipo fundamental de reconhecedor de linguagens regulares, largamente utilizado na compilação e, em particular, no projeto de blocos léxicos (SCHEFFEL). Eles classificam-se fundamentalmente em:

- Autômato Finito Determinístico (AFD): Possui uma função de transição onde o controle passa para um único outro estado dado um símbolo de entrada (SCHEFFEL).
- Autômato Finito Não-Determinístico (AFND): A sua função de mapeamento permite que um mesmo estado, ao ler um símbolo, direcione o controle simultaneamente para múltiplos estados (SCHEFFEL).

3.2. Determinização

Os modelos determinísticos e não-determinísticos representam a mesma classe de linguagens, de modo que para todo AFND é possível construir um AFD equivalente (SCHEFFEL). O processo de determinização consiste em mapear cada novo estado do AFD como um subconjunto de estados do AFND de origem, aglomerando os caminhos simultâneos criados pela falta de determinismo.

3.3. Minimização

Conforme Scheffel, um AFD é considerado mínimo quando obedece a três regras: 1) não possui estados inacessíveis; 2) não possui estados mortos; e 3) não possui estados equivalentes.

- Estados Inacessíveis: São aqueles que não podem ser alcançados por nenhuma cadeia a partir do estado inicial.
- Estados Mortos: São estados que não são finais e, a partir dos quais, nenhum estado final pode ser alcançado.
- Estados Equivalentes: São estados agrupados em uma mesma Classe de Equivalência.

4. Especificação e Implementação da Solução

A aplicação foi desenvolvida em linguagem Python, tirando proveito de suas estruturas de dados embutidas, que se adequam naturalmente à Teoria dos Grafos e Conjuntos. O autômato foi modelado através de uma Classe, onde as transições foram especificadas por meio de dicionários aninhados (dict), e o alfabeto e conjunto de estados foram estruturados através de conjuntos matemáticos (set), que evitam intrinsecamente a duplicação de elementos.

O ciclo de vida da aplicação foi estruturado em quatro etapas principais:

1. Carga e Geração do AFND: O sistema realiza o processamento do texto (leitura de tokens e de regras em notação BNF). Cumprindo os requisitos de especificação, apenas o estado inicial (S) é compartilhado; todas as derivações subsequentes instanciam estados exclusivos (ex: q1, q2 ...). O resultado dessa carga é um autômato altamente não-determinístico.
2. Determinização (Construção de Subconjuntos): A função determinar implementa a teoria descrita por Scheffé iterando sobre os agrupamentos de destinos alcançados. Quando o estado S contém duas transições com o mesmo símbolo para caminhos distintos, o algoritmo funde ambos gerando um novo estado de subconjunto (ex: q1,q2), eliminando a multiplicidade de rotas para a mesma letra.
3. Minimização (Busca e Poda de Grafos): Devido à restrição de não aplicar classes de equivalência no processo, a minimização foi resolvida através de algoritmos de travessia. Primeiramente, uma busca em largura (BFS) a partir de S mapeia todos os vértices atingíveis, podando os estados inacessíveis. Posteriormente, uma nova varredura a partir de cada estado restante verifica a existência de caminhos viáveis até o conjunto de estados finais, expurgando os estados mortos.
4. Tratamento de Exceções (Estado de Erro): A matriz minimizada resultante é percorrida para o mapeamento final. Em todas as coordenadas de estados e símbolos do alfabeto onde não havia transição válida previamente mapeada, injeta-se o apontamento para o estado especial de Erro. Este estado final age como um poço infinito, direcionando todas as entradas subsequentes para si mesmo.

5. Conclusão

A implementação cumpriu todos os requisitos propostos, convertendo satisfatoriamente um conjunto de gramáticas regulares e cadeias lexicais em um Autômato Finito Determinístico robusto. A exclusão intencional das classes de equivalência exigiu uma abordagem algorítmica puramente focada no rastreamento e utilidade dos caminhos, o que demonstrou na prática o comportamento dos estados inalcançáveis e mortos definidos na literatura. A ferramenta desenvolvida prova-se capaz de compor o bloco léxico elementar para análises mais profundas, processando erros e caminhos válidos deterministicamente.

6. Referencial Bibliográfico

- SCHEFFEL, Roberto M. Apostila de Linguagens Formais e Autômatos. Departamento de Ciências Tecnológicas, Curso de Ciência da Computação, UNISUL - Universidade do Sul de Santa Catarina.